

Digital Image Processing - Lab Report 3

Multiresolution Analysis & Perfect Reconstruction Filter Banks

Name: Georgia Papachristodoulou
AEM: 11177

Introduction

The objective of this assignment is to design and implement a Multiresolution Analysis (MRA) framework for digital images. The project entails the construction of strict Perfect Reconstruction (PR) orthonormal filter banks and the development of 1D and separable 2D analysis and synthesis algorithms. By utilizing the Daubechies 10 (db10) wavelet filter, the implemented system decomposes a grayscale image into distinct frequency sub-bands (LL, LH, HL, HH) and subsequently reconstructs it. The ultimate goal is to practically demonstrate the mathematical principles of zero-distortion and zero-aliasing in multirate signal processing, achieving a flawless reconstruction of the original signal.

1. Orthonormal Filter Banks Generation

The mathematical constraints for perfect reconstruction in orthonormal filter banks (as per Equations 14-17 of the assignment) require strict time reversals (e.g., $G_0(z) = H_0(z^{-1})$) and extensive temporal shifts (e.g., $G_1(z) = -z^{-2K+1}G_0(-z^{-1})$). If the base low-pass filter $h_0[n]$ has a length of K and is defined on the interval $n \in [0, K - 1]$, the analytical expansion dictates that $g_0[n]$ exists in $[-(K - 1), 0]$, $g_1[n]$ exists in $[2K - 1, 3K - 2]$, and $h_1[n]$ exists in $[-(3K - 2), -(2K - 1)]$.

To strictly adhere to the assignment's prerequisite regarding the placement of appropriate zeros, all four impulse responses were embedded into identically sized, zero-padded NumPy arrays. The global time axis was calculated to span from $n_{min} = -(3K - 2)$ to $n_{max} = 3K - 2$. A helper index-mapping function was utilized to shift the true time coordinate n to the 0-indexed Python array coordinates. This critical alignment guarantees that $n = 0$ lies at the exact same array index for all filters, completely preventing catastrophic phase misalignments during the subsequent convolution stages.

```
1 def orthonormalFilterBanks(h: np.ndarray):
2     K = len(h)
3     n_min = -(3 * K - 2)
4     n_max = 3 * K - 2
5     total_length = n_max - n_min + 1
6
7     h0 = np.zeros(total_length)
8     h1 = np.zeros(total_length)
9     g0 = np.zeros(total_length)
10    g1 = np.zeros(total_length)
11
12    def idx(n):
13        return n - n_min
14
15    for n in range(0, K):
16        h0[idx(n)] = h[n]
17
```

```

18     for n in range(-(K-1), 1):
19         g0[idx(n)] = h[-n]
20
21     for n in range(2*K - 1, 3*K - 1):
22         g1[idx(n)] = -((-1) ** (n - 2*K + 1)) * h[n - 2*K + 1]
23
24     for n in range(-(3*K - 2), -(2*K - 1) + 1):
25         h1[idx(n)] = g1[idx(-n)]
26
27     return h0, h1, g0, g1

```

Listing 1: Python Implementation of orthonormalFilterBanks

2. 1D Analysis and Synthesis

The `analysis1d` and `synthesis1d` functions constitute the core computational engine for the 1D Multiresolution Analysis (MRA) framework.

For the **analysis stage**, the input 1D signal is independently convolved with the analysis filters h_0 (low-pass) and h_1 (high-pass). A critical implementation detail is the utilization of `scipy.signal.convolve` with the parameter `mode='full'`. This ensures that all transient boundary points are preserved, which is an absolute mathematical prerequisite for perfect reconstruction. The filtered signals are subsequently downsampled by a factor of 2, retaining only the even-indexed samples.

In the **synthesis stage**, the process is inverted. The two sub-band signals are first upsampled by a factor of 2 by inserting a zero between every consecutive sample. The upsampled signals w_0 and w_1 are then convolved with their respective synthesis filters g_0 and g_1 (again utilizing `mode='full'`). Finally, the reconstructed signal $\hat{x}$ is obtained by performing an element-wise addition of the two filtered components.

```

1 def analysis1d(h0: np.ndarray, h1: np.ndarray, x: np.ndarray):
2     v0 = convolve(x, h0, mode='full')
3     v1 = convolve(x, h1, mode='full')
4     y0 = v0[::2]
5     y1 = v1[::2]
6     return y0, y1
7
8 def synthesis1d(g0: np.ndarray, g1: np.ndarray, y0: np.ndarray, y1: np.ndarray):
9     :
10    w0 = np.zeros(2 * len(y0))
11    w1 = np.zeros(2 * len(y1))
12    w0[::2] = y0
13    w1[::2] = y1
14
15    r0 = convolve(w0, g0, mode='full')
16    r1 = convolve(w1, g1, mode='full')
17    xhat = r0 + r1
18    return xhat

```

Listing 2: Python Implementation of 1D Analysis and Synthesis

3. 2D Analysis and Synthesis (Separability)

The extension of Multiresolution Analysis (MRA) to two-dimensional signals (images) is achieved separably, utilizing the previously established 1D principles. This separable approach drastically reduces computational complexity from $O(K^2)$ to $O(2K)$.

In the `analysis2d` function, the image is decomposed in two sequential phases. First, the 1D analysis filters h_0 and h_1 are reshaped into column vectors $(K, 1)$ and convolved vertically

with the input image, followed by a 2:1 vertical downsampling (rows). In the second phase, the horizontal components are processed: the intermediate vertical sub-bands are convolved with the filters reshaped into row vectors $(1, K)$, followed by horizontal 2:1 downsampling (columns). This highly optimized, vectorized approach correctly partitions the image into four distinct sub-bands $(y_{00}, y_{01}, y_{10}, y_{11})$.

The `synthesis2d` function mirrors this process in reverse to achieve perfect reconstruction. Phase 1 performs horizontal synthesis: it upsamples the sub-bands by inserting zero-columns, filters them along the rows using the reshaped 1D synthesis filters g_0 and g_1 , and adds the components. Phase 2 executes vertical synthesis by upsampling the intermediate results along the rows (inserting zero-rows), filtering them vertically, and finally superimposing them to yield the fully reconstructed image $\hat{x}$.

4. Demo Execution & Results Commentary

The final script consolidates the multiresolution analysis pipeline. The grayscale image `farm.jpg` was loaded and its maximum intensity L was extracted for normalization purposes. The base low-pass filter coefficients for the Daubechies 10 (`db10`) wavelet were retrieved using the `pywt` (PyWavelets) library. The `db10` filter was selected because it is an orthonormal filter that features limited support in both the spatial and frequency domains, making it ideal for image analysis.

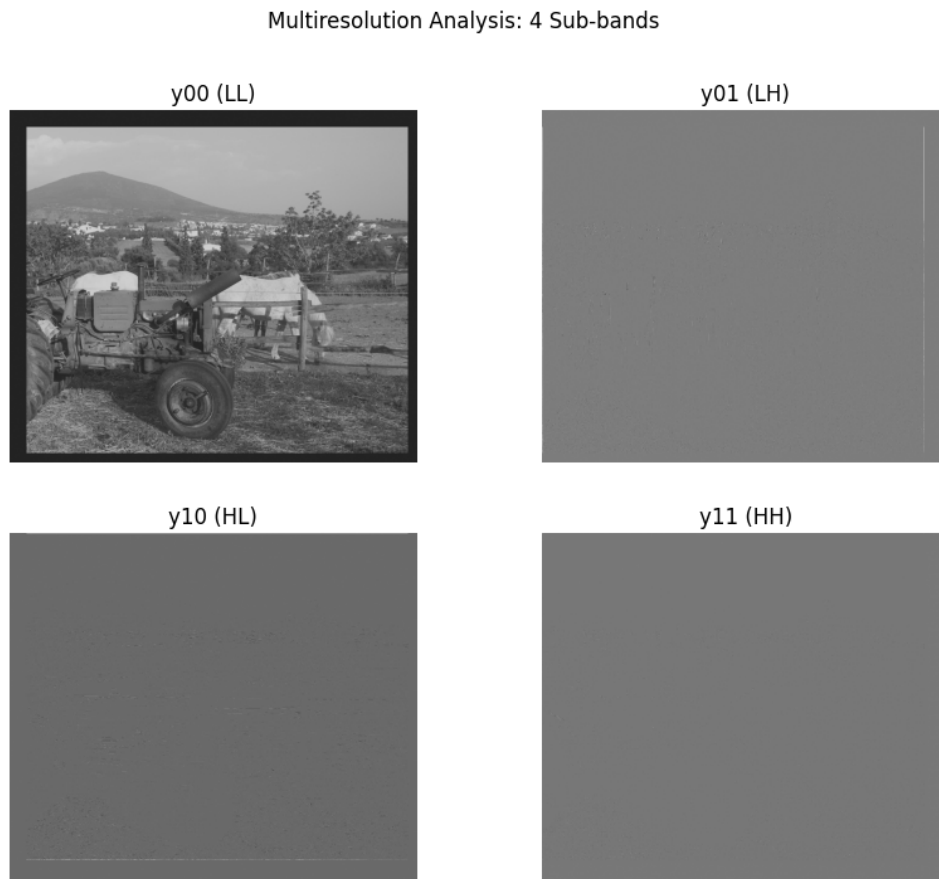


Figure 1: Multiresolution Analysis: 4 Sub-bands

Sub-bands Commentary: The `analysis2d` function successfully decomposed the image into four distinct sub-bands (Figure 1).

- y_{00} (**LL**): Contains the low-frequency approximation of the image, essentially serving as a downsampled thumbnail of the farm.
- y_{01} (**LH**): Captures the horizontal high-frequency details (e.g., the horizontal wooden beams of the fence).
- y_{10} (**HL**): Isolates the vertical details (e.g., the vertical fence posts and tree trunks).
- y_{11} (**HH**): Captures diagonal high-frequency noise and structural corners.

(Note: The grey background in the LH, HL, and HH sub-bands represents values close to zero, effectively highlighting the positive and negative high-frequency edge variations).



Figure 2: Perfect Reconstruction Comparison

Reconstruction Commentary: The four sub-bands were passed into the `synthesis2d` function. Because the convolution with `mode='full'` inherent to the filtering process introduces

transient padding and spatial delays, a central crop was applied to the output $\hat{x}$ to geometrically align it with the original image dimensions. As demonstrated in Figure 2, the reconstructed image $\hat{x}$ is visually identical to the original input x . This flawless visual match empirically validates that the zero-padding boundary treatment, the strict temporal alignments established in the orthonormal filter generation, and the symmetric separable 2D processing logically combined to successfully satisfy the Perfect Reconstruction conditions.